Smart Parking System — Research Document

Stage 3: research, simulation and documentation. Sending every arriving car to the nearest free stall with A*: an ESP32 prototype on a 1:100 model and a simulation study of what it saves.

Course	A Project in Internet of Things (IoT) — Technion, Faculty of Computer Science (ICST)
Authors	Aseel Kurdy, Nayef Habiballah, Osama Abdulhalim
Instructors	Itai Dabran, Ido Ram, Diana Cohen
Date	25/10/2026
Repository	github.com/AseelKurdy/smart-parking-lot (firmware, tests, simulation, documents)
Demo video	release v1.0 — SmartParking_demo.mp4 (3 min)

1. General description

Drivers who look for a parking space on their own drive further, burn more fuel and lose time. We built a smart parking system that **assigns every arriving car the nearest free stall**, tells the driver where to go, and checks whether the car parked where it was sent. The prototype is a two-floor, 14-stall parking lot built at a scale of 1:100 and run by a single ESP32 board. An ultrasonic sensor across the entrance counts arriving cars, a second ultrasonic sensor above one stall confirms real parking, and an LED tells the driver whether they are in the right stall. The lot is stored as a road network with the lengths measured on the model, and the A* algorithm finds the shortest driving route to every stall. A reserved stall is held for 60 seconds. The board creates its own Wi-Fi network and serves a live 2D/3D map of the lot to any phone or laptop.

To find out whether the idea pays off in a real garage, the same assignment rule was evaluated in a simulation of a 216-stall, three-floor garage with arrival and parking-time data from published studies. At 85% occupancy, assigning the stall saves **85 m of driving per car (31 s, 8.5 mL of fuel, 20 g of CO_2)** compared with a driver who searches aisle by aisle and notices every free stall; over a year this is about 4,700 driver-hours, 4,700 litres of fuel, 11 tonnes of CO_2 and ₪166,000. The firmware itself was verified with 54 automated tests on a PC simulator of the board.

2. Introduction

The need. Searching for parking is a measurable waste. A review of 16 studies found that drivers cruised for parking for 8.1 minutes on average and that cruising made up about 30% of traffic in the congested downtown areas studied (Shoup, 2006). Inside off-street facilities the search is shorter but still real: drivers reported an average search of 50 s off-street against 30 s on-street in a survey of 11,425 trips (van Ommeren, Wentink & Rietveld, 2012), and a GPS study measured a mean of 1.5 minutes of cruising (Saki & Hagen, 2024). Search effort also rises sharply with occupancy: it becomes noticeable above about 85% occupancy and exceeds a minute above 92–93% (Levy, Martens & Benenson, 2013).

Solutions today. Published systems attack the problem in two ways. *Guidance and information systems* tell drivers where free spaces are: zone-level availability information cut search time in a parking facility by 16.2% in simulation (Caicedo, 2009), a smart parking system saved 11 s of search per car in a university lot (Surpris, Liu & Vincenzi, 2014), and San Francisco's sensor-based SFpark pilot reported that the average search time in its pilot areas fell by 43% against control areas (SFMTA, 2014). *Assignment and reservation systems* go further and give each driver a specific space: allocating and reserving spaces roughly halved the time to park in simulation (Geng & Cassandras, 2011). Surveys of the field describe the same split between information collection, system deployment and service dissemination (Lin, Rivano & Le Mouël, 2017).

What we propose. A self-contained system in the second category, small enough to run on one microcontroller: the lot itself decides where each car should go, using the real driving distances of the lot, holds the stall for the driver for a limited time, detects drivers who park elsewhere and immediately re-balances the reservations, and shows everything live on a web page served by the board — with no server, cloud or internet connection. Figure 1 shows the model and the live view.

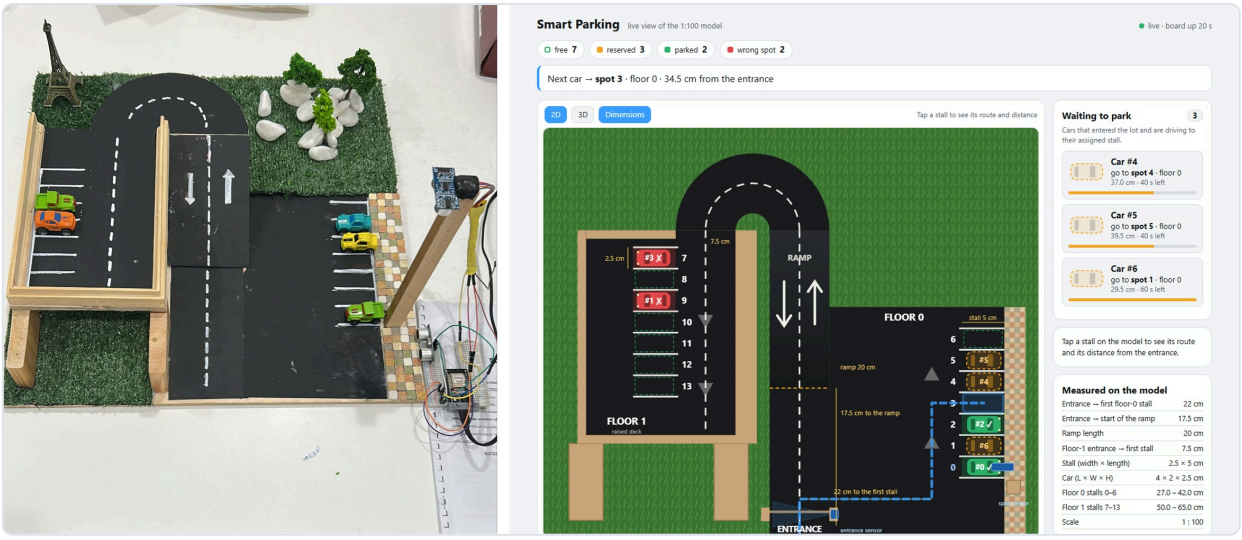


Figure 1. Left: the 1:100 model — floor 1 on the raised deck (left), the ramp with its U-turn, floor 0 (right), the entrance sensor on its post and the ESP32 board. Right: the live web view served by the board.

3. Related work

Smart parking as a field. Smart parking grew from simple car-park counting and guidance signs into networked systems that sense individual spaces and share the data with drivers. Idris et al. (2009) reviewed the early generation of such systems and their technologies, from parking guidance and information (PGI) systems to reservation and e-parking concepts, with a focus on how vehicles are detected. Lin, Rivano & Le Mouël (2017) surveyed the literature of 2000–2016 and organised it into three themes: *information collection* (sensors and crowd-sourcing), *system deployment* (models, allocation, pricing and reservation) and *service dissemination* (how results reach drivers). Al-Turjman & Malekloo (2019) reviewed smart parking in IoT-enabled cities, classifying systems by their hardware and software design factors and highlighting data reliability, security and privacy as open issues. Our project combines all three of Lin et al.'s themes in one device: it collects occupancy data with sensors, runs an allocation model, and disseminates the result through a web page.

Allocation and reservation. Most deployed systems only report free spaces and leave the choice to the driver. Geng & Cassandras (2011) formulated parking as an optimal resource-allocation problem with reservations and showed in simulation that assigning spaces roughly halves the time to park. Caicedo (2009) showed that even coarse, zone-level availability information shortens the search. Real deployments such as SFpark (SFMTA, 2014) combined space-level sensors with demand-responsive pricing and reported less searching and less circling traffic. Our system follows the assignment approach: the nearest stall by actual driving distance, held for a limited time.

Sensing technologies. Vehicle-detection sensors fall into *intrusive* sensors installed in or under the road surface (inductive loops, magnetometers) and *non-intrusive* sensors mounted above or beside the space (ultrasonic, infrared, microwave radar, cameras) (Idris et al., 2009). In-ground sensors are robust but need roadworks; cameras cover many spaces at once but need image processing, lighting and raise privacy questions; ultrasonic sensors are cheap, simple and well suited to indoor garages, where they are commonly mounted above each space. For a 1:100 model with a microcontroller, an ultrasonic range sensor is the natural choice: it measures a distance directly, needs no calibration other than thresholds, and works in a closed structure.

The sensor we used: HC-SR04. The HC-SR04 is an ultrasonic ranging module powered from 5 V (about 15 mA). A 10 µs pulse on its *Trig* input makes it send a burst of eight 40 kHz ultrasonic pulses; it then raises its

Echo output for as long as the sound takes to travel to the nearest object and back. The distance is half the round-trip time multiplied by the speed of sound (about 343 m/s at 20 °C), which the datasheet gives as *distance [cm] = echo time [µs] / 58*. The module measures 2–400 cm with a resolution of about 3 mm within a measuring angle of about 15°, and the datasheet recommends a measurement cycle of at least 60 ms so that an echo cannot be confused with the next burst (ElecFreaks, n.d.). These figures shaped our design: the two sensors are fired 60 ms apart; readings below the 2 cm minimum are treated as electrical noise; and the 3 mm resolution sets the limit on how close our thresholds can be (Section 4.3).

4. Project description

4.1 What we built

The **model** is a 1:100 parking lot on a wooden base (Figure 1): black card roads with painted lane markings and stalls, artificial grass, a raised wooden deck for floor 1, and a ramp with a U-turn between the floors. Every length the software uses was measured on the model: a stall is 2.5 × 5 cm (a standard 2.5 × 5 m bay), a car 4 × 2 × 2.5 cm, the floor-0 lane starts 22 cm from the entrance, the ramp starts 17.5 cm from the entrance and is 20 cm long, and the first floor-1 stall is 7.5 cm past the ramp.

The **electronics** are an ESP32 development board (dual-core 240 MHz microcontroller with built-in Wi-Fi; Espressif Systems, 2023), two HC-SR04 sensors and the board's built-in LED. The firmware is written in C++ with the Arduino core for ESP32, version 3.0.1 (Espressif Systems, n.d.).

Function	Device	Trig	Echo
Entrance detection (sideways across the road)	HC-SR04	GPIO 27	GPIO 14
Stall 0 detection (from above)	HC-SR04	GPIO 26	GPIO 25
Correct / wrong parking indicator	built-in LED	—	GPIO 2

One stall (stall 0) has a physical sensor; the other 13 stalls report parking through a serial command, which demonstrates the full logic with one sensor per stall where a real lot would have one sensor in every stall.

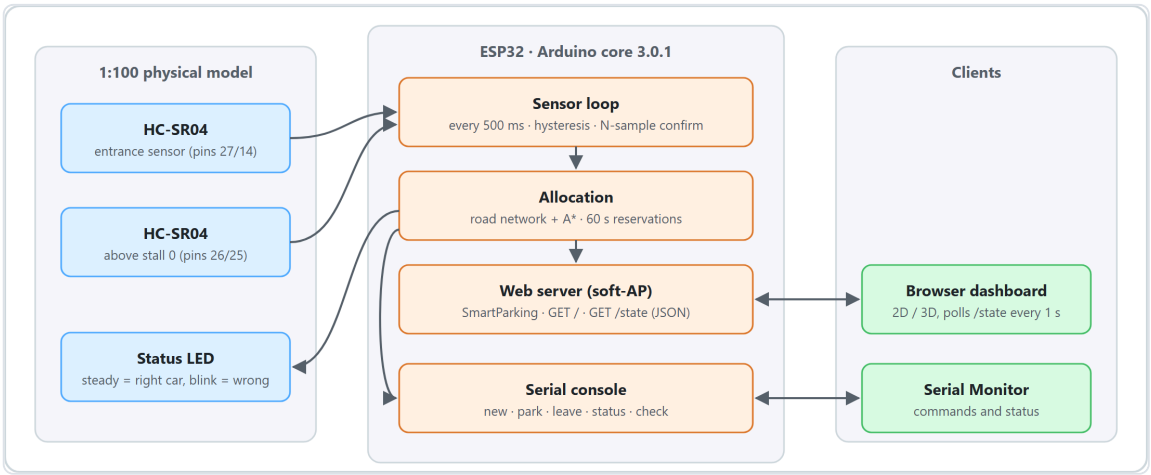


Figure 2. System architecture: sensors and LED on the model, the four parts of the firmware on the ESP32, and the two clients.

4.2 Communication

- **Wi-Fi.** The ESP32 runs as a Wi-Fi access point named *SmartParking*, so no router or internet is needed. It serves an HTTP server on port 80 with two endpoints: `GET /` returns the web page (HTML, CSS and

JavaScript stored in the board's flash memory) and `GET /state` returns the full state of the lot as JSON — every stall's state, the car in it, the waiting cars with their remaining reservation time, and the measured geometry.

- **Live view.** The page in the phone's or laptop's browser requests `/state` once a second and redraws the lot in 2D or 3D: free, reserved (with countdown), parked correctly, wrong stall and blocked stalls, the route the next car will take, and a waiting list.
- **Serial.** Over USB (9600 baud) the board prints every event and accepts commands: `new`, `park <stall> <car>`, `leave <car>`, `status` and `check [seconds]` (raw sensor readings for calibration).

4.3 Algorithms

Road network and A*. The lot is a graph built at start-up from the measured lengths (Figure 3): the entrance, the start of the ramp, the floor-1 entrance, a lane point in front of each stall (2.5 cm apart — one stall width) and the 14 stalls, each hanging off its lane point by its length (5 cm). A stall can only be a destination, never driven through. The driving distance to every stall is found with A* (Hart, Nilsson & Raphael, 1968), a best-first search that expands the node with the smallest $f = g + h$, where g is the distance driven so far and h an estimate of the remaining distance. Our heuristic is the stall length (every route ends by driving into the stall) plus, on the goal's own lane, the remaining distance along the lane. It never overestimates the true distance, so A* returns the shortest path, as Dijkstra's algorithm would (Dijkstra, 1959), while expanding fewer nodes. Because the lot is fixed, the 14 distances are computed once at boot (27.0–42.0 cm on floor 0, 50.0–65.0 cm on floor 1), and assigning a stall at run time is a lookup.

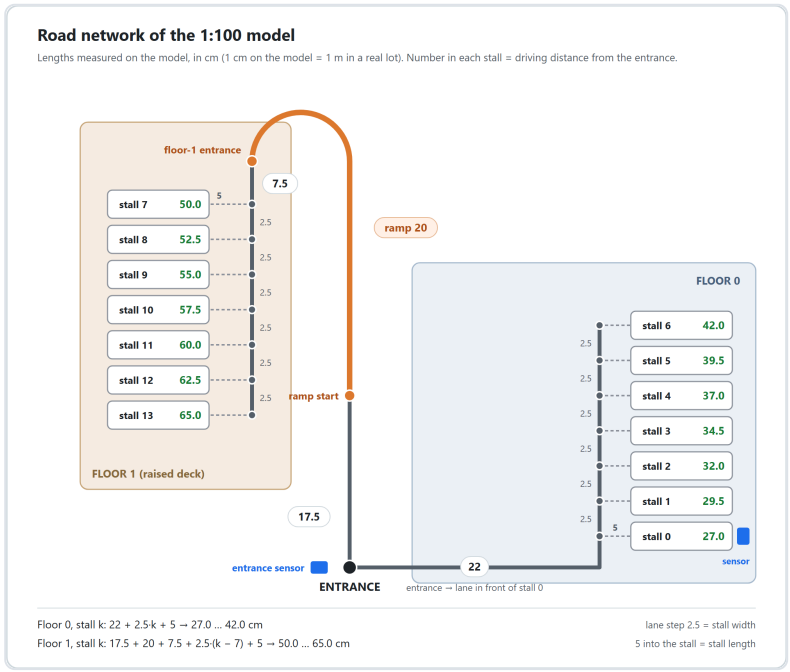


Figure 3. The road network drawn in the model's real layout, with the measured lengths and every stall's driving distance from the entrance.

Assignment rules. A new car gets the nearest stall that is free, not reserved for another car and not blocked, and the stall is reserved for 60 seconds. If the car parks in its stall it is *correct*. If it parks in another free stall, its own reservation is released at once and the stall is marked *wrong*. If it takes a stall reserved for another car, that car is immediately re-routed to the nearest free stall. A reservation that expires returns the stall to the pool. An object detected on stall 0 with no car on its way marks the stall *blocked* so nobody is sent there.

Sensor processing. Both sensors are read every 500 ms. The entrance sensor reads about 30 cm (the wall opposite) when the road is empty and about 22 cm with a car; a car is counted below 25 cm after two consecutive readings, and only once until the road reads empty twice — a car that stops in front of the sensor is not counted again. The stall-0 sensor reads 25–26 cm when the stall is empty and 22.5–24.5 cm with a car (depending on its height, 2.5 or 1.5 cm). It uses hysteresis — a car arrives below 24.7 cm and leaves above 24.9 cm — and a change of state needs four consecutive valid readings (2 s), so a reading near a threshold cannot make the state flicker. Readings with no echo or below the sensor's 2 cm minimum are invalid and restart the count; this removed phantom cars caused by electrical noise on the Echo line, which our automated tests found on the real board.

5. Simulation description

What we checked and how. The question was how much driving, time, fuel, CO_2 and money the assignment rule saves compared with drivers who search on their own. We wrote a discrete-event simulation in Python that follows individual cars through a parking lot (Appendices A and B). Cars arrive as a Poisson process and stay for a gamma-distributed time with a mean of 88 minutes, both taken from a study of a mall garage (Huang et al., 2020); the arrival rate for each target occupancy is found from the Erlang loss model of a lot with c stalls. Three strategies are applied to exactly the same cars (common random numbers): **assigned** — the firmware's rule, the nearest free stall by A*; **searching alone** — a driver who searches aisle by aisle, as people do, noticing each free stall with probability p (p = 1, the most favourable case for the driver, and p = 0.5); and **random search**, as an upper bound. Two lots were simulated: the prototype's 14-stall lot and the main scenario, a 216-stall, three-floor garage with three two-way aisles per floor, built to the dimensions of the Israeli parking design guidelines (Israel Ministry of Transport, 2000). Each setting was run at eight occupancy levels from 50% to 98%, with 20 replications of 3,000 measured cars after a warm-up. Distances are converted to time, fuel, CO_2 and money with sourced constants (Appendix C). Eleven automated tests check the simulation, among them that A* agrees with an exhaustive Dijkstra search for every stall and that the simulated occupancy matches the Erlang formula.

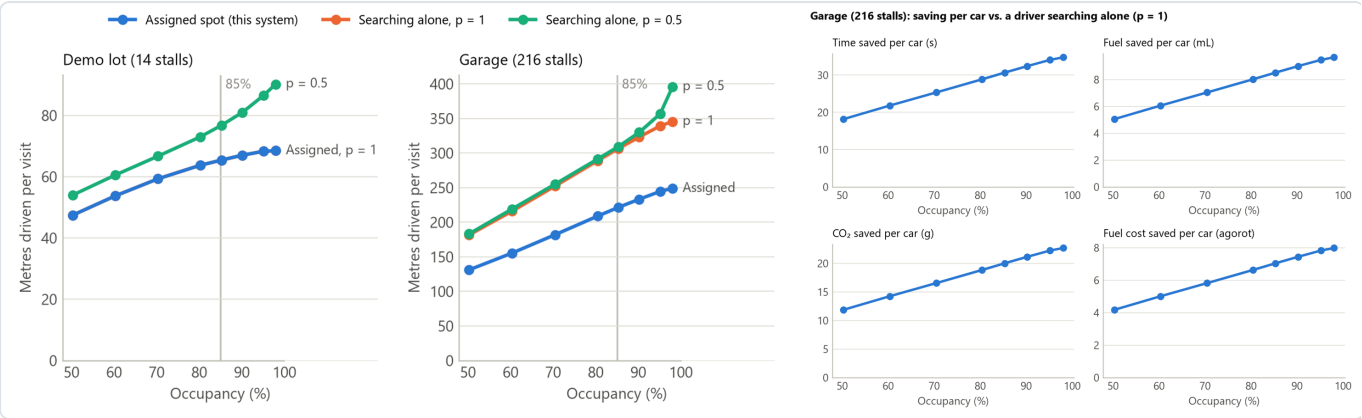


Figure 4. Left: metres driven inside the lot per visit against occupancy, for both lots. Right: savings per car in the garage compared with a driver searching alone (p = 1). Shaded bands are 95% confidence intervals.

Results. In the garage, the assigned driver always drives less than the driver searching alone, and the gap grows as the garage fills (Figure 4, left): at 85% occupancy the assigned driver drives 222 m per visit and the searching driver 307 m. Being assigned cuts the driving to the stall by 43–44% at every occupancy level. The saving per car grows from 18 s at 50% occupancy to 35 s at 98%, and to 53 s at 98% when the driver misses half of the free stalls (Figure 4, right).

Garage (216 stalls), 85% occupancy, assigned vs. searching alone (p = 1)	Per car	Per year
Driving saved	85.2 ± 1.1 m	46,675 km
Time saved	30.7 s	4,667 h
Fuel saved	8.5 mL	4,667 L
CO_2 avoided	20.0 g	11.0 t
Money value (fuel + drivers' time)	₪0.30	₪166,069

How the project helps. Per car the saving is small, but a garage of this size serves about 548,000 cars a year, so seconds and millilitres add up to thousands of hours and litres. The result is a lower bound: the searching driver is modelled favourably (an efficient route, every free stall noticed, no circling), and it agrees with published figures without being tuned to them — 31 s of extra search is below the 50 s reported for off-street parking (van Ommeren et al., 2012), and the 43–44% reduction lies inside the 16–50% range reported for information and reservation systems (Caicedo, 2009; Geng & Cassandras, 2011). The benefit comes from routing drivers past full aisles and floors: in the prototype's small single-lane lot, a driver who notices every free stall already finds the nearest one, so the system helps there only when drivers miss stalls (11.3 m per car at p = 0.5) or when the lot is full. The largest uncertainty is the driving speed inside the garage; the one-at-a-time sensitivity analysis is in Appendix C.

The firmware was checked in simulation too. The unmodified firmware was compiled on a PC with simulated sensors, serial port, LED and web server, and an automated suite of 54 tests (parser, allocation rules, A* distances against an independent formula, timeouts, sensor thresholds, web data) passed in full; a model-based test compared the firmware with an independent model of the rules after each of 161,040 random operations with no difference. The behaviour on the real board was demonstrated in the recorded video.

6. Conclusions

- A working ESP32 prototype on a measured 1:100 model counts arriving cars, assigns the nearest free stall by real driving distance using A*, holds it for 60 seconds, detects correct and wrong parking with a sensor and an LED, keeps reservations consistent when drivers do not follow them, and shows the lot live on a web page served by the board itself.
- In a simulated 216-stall garage at 85% occupancy, assigning the stall saves 85 m, 31 s, 8.5 mL of fuel and 20 g of CO_2 per car — about ₪166,000 a year in fuel and drivers' time — and the saving grows with occupancy. Because the searching driver is modelled favourably, this is a lower bound, and it agrees with published figures.
- The benefit is largest where the order in which a driver searches differs from the shortest path: garages with several aisles and floors. In a small single-lane lot it is limited to drivers who miss free stalls and to cars that would otherwise search a full lot.
- Next steps: a sensor in every stall instead of one physical stall, sturdier sensor wiring (separate Trig/Echo wires, a decoupling capacitor and a voltage divider on the 5 V Echo line), and simulating daily arrival peaks and drivers who ignore their assignment.

References

- Al-Turjman, F., & Malekloo, A. (2019). Smart parking in IoT-enabled cities: A survey. *Sustainable Cities and Society, 49*, 101608. https://doi.org/10.1016/j.scs.2019.101608
- Caicedo, F. (2009). The use of space availability information in "PARC" systems to reduce search times in parking facilities. *Transportation Research Part C, 17*(1), 56–68. https://doi.org/10.1016/j.trc.2008.07.001
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*, 269–271. https://doi.org/10.1007/BF01386390
- ElecFreaks. (n.d.). *Ultrasonic ranging module HC-SR04* [Datasheet]. https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf
- Espressif Systems. (2023). *ESP32 series datasheet* (v4.x). https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Espressif Systems. (n.d.). *Arduino-ESP32 documentation* (Wi-Fi and WebServer libraries). https://docs.espressif.com/projects/arduino-esp32/en/latest/
- Geng, Y., & Cassandras, C. G. (2011). A new "smart parking" system based on optimal resource allocation and reservations. *Proc. 14th IEEE ITSC*. https://people.bu.edu/cgc/Published/ITSC11.pdf
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136
- Huang, X., Long, X., Wang, J., & He, L. (2020). Research on parking sharing strategies considering user overtime parking. *PLOS ONE, 15*(6), e0233772. https://doi.org/10.1371/journal.pone.0233772
- Idris, M. Y. I., Leng, Y. Y., Tamil, E. M., Noor, N. M., & Razak, Z. (2009). Car park system: A review of smart parking system and its technology. *Information Technology Journal, 8*(2), 101–113. https://doi.org/10.3923/itj.2009.101.113
- Israel Ministry of Transport. (2000). *Parking planning guidelines, Chapter D: Planning parking facilities*. https://www.gov.il/BlobFolder/policy/parking_planning_guidelines/he/parking.pdf
- Levy, N., Martens, K., & Benenson, I. (2013). Exploring cruising using agent-based and analytical models of parking. *Transportmetrica A, 9*(9), 773–797. https://doi.org/10.1080/18128602.2012.664575
- Lin, T., Rivano, H., & Le Mouël, F. (2017). A survey of smart parking solutions. *IEEE Transactions on Intelligent Transportation Systems, 18*(12), 3229–3253. https://doi.org/10.1109/TITS.2017.2685143
- Saki, S., & Hagen, T. (2024). Cruising for parking again. *Transportation Research Part A, 183*. https://doi.org/10.1016/j.tra.2024.104045
- SFMTA — San Francisco Municipal Transportation Agency. (2014). *SFpark pilot project evaluation*. https://www.sfmta.com/getting-around/drive-park/demand-responsive-pricing/sfpark-evaluation
- Shoup, D. C. (2006). Cruising for parking. *Transport Policy, 13*(6), 479–486. https://doi.org/10.1016/j.tranpol.2006.05.005
- Surpris, G., Liu, D., & Vincenzi, D. (2014). How much can a smart parking system save you? *Ergonomics in Design, 22*. https://doi.org/10.1177/1064804614526202
- van Ommeren, J. N., Wentink, D., & Rietveld, P. (2012). Empirical evidence on cruising for parking. *Transportation Research Part A, 46*(1), 123–130. https://doi.org/10.1016/j.tra.2011.09.011

Sources of the simulation constants (fuel, CO_2, prices, value of time) are listed with each constant in Appendix C.

Appendix A — Simulation user guide

A.1 What the simulation does

The simulation estimates how much driving, time, fuel, CO_2 and money is saved when every arriving car is sent to the nearest free stall (this project's rule), compared with drivers who search on their own. It runs two lots — the prototype's 14-stall lot and a 216-stall, three-floor garage — at eight occupancy levels, and writes the results as tables and figures.

A.2 Requirements and installation

- Python 3.8 or newer (tested with Python 3.12.10).
- Three libraries: `numpy`, `pandas` and `matplotlib` (tested with numpy 2.5.4, pandas 3.1.0, matplotlib 3.11.2).
- The code: folder `simulation/` of the repository github.com/AseelKurdy/smart-parking-lot (*Code* → *Download ZIP*, or `git clone`).

```
pip install numpy pandas matplotlib
```

A.3 Running it

All commands run from inside the `simulation/` folder.

Command	What it does	Time
`python test_sim.py`	Runs the 11 automated checks (A* vs. Dijkstra, the Erlang formula, reproducibility …). All must print `OK`.	≈ 6 s
`python run_all.py`	Runs every experiment and writes `results/*.csv` and `figures/*.png` (existing figures are replaced).	≈ 3 min
`python run_all.py --figures`	Redraws the figures from the saved results without running the simulation again.	seconds

The same random seeds are used every time, so a run reproduces the published results exactly.

A.4 Outputs

File	Contents
`results/headline.csv`	The main result: per lot and baseline at 85% occupancy — metres, seconds, fuel, CO_2 and money saved per car and per year.
`results/occupancy_sweep.csv`	Mean and 95% confidence interval of every measure, per lot, *p* and occupancy level.
`results/occupancy_sweep_replications.csv`	The same, one row per replication (raw data).
`results/parameters.csv`	Every constant used, with its source, page and sensitivity range.
`results/sensitivity.csv`	The yearly saving when each uncertain constant is set to its low and high value.
`results/validation.csv`	The model's search time and driving reduction next to the published benchmark figures.
`figures/fig1_driving_vs_occupancy.png`	Metres driven per visit against occupancy (Figure 4, left).
`figures/fig2_savings_vs_occupancy.png`	Savings per car against occupancy (Figure 4, right).
`figures/fig3_sensitivity.png`	Sensitivity of the yearly saving (Appendix C).

A.5 Changing the scenario

- **A constant** (driving speed, fuel use, garage size, parking duration …): change its `value` in `params.py`. Each constant keeps its source next to it; uncertain ones also have a `sweep` range used by the sensitivity analysis.
- **The occupancy levels, replications or cars per replication:** `OCCUPANCIES`, `REPS` and `CARS` at the top of `run_all.py`.
- After a change, run `python test_sim.py` and then `python run_all.py`.

Appendix B — Simulation programmer guide

B.1 Structure

File	Role
`params.py`	Every constant (`PARAMS`), each a `Param(value, unit, source, where, note, verified, sweep)` ; `derived()` computes the step costs and the ramp length from them; `BENCHMARKS` are published figures used only to check the results; `REFERENCES` the sources. Single source of truth: nothing else hard-codes a number.
`lot.py`	Lot geometry and path finding. `Lot` stores a lot as text grids (`.` lane, `P` stall, `E` entrance, `R` ramp, `#` blocked) with step costs; `a_star()` is the firmware's A* rule; `dijkstra()` checks it; `demo_lot()` builds the 14-stall lot and `garage()` the multi-storey garage with its search route.
`sim.py`	The simulation. `LotModel` precomputes, per lot, each stall's A* distance, the order in which a searching driver passes the stalls, and the distances between stalls. The strategies `assigned()` , `cruising()` and `random_search()` take the current occupancy and return the chosen stall and the metres driven. `simulate()` runs one replication; `erlang_b()` and `arrival_rate_for()` give the arrival rate for a target occupancy; `to_costs()` converts metres to seconds, fuel, CO_2 and money.
`run_all.py`	The experiments: `build_models()` , `run()` (all replications of one setting), `aggregate()` (means and 95% confidence intervals), the yearly totals, the sensitivity analysis and the three figures; `main()` writes everything to `results/` and `figures/` .
`test_sim.py`	Eleven `unittest` checks of the geometry, A*, the search route, the Erlang formula, the occupancy calibration and reproducibility.

B.2 How a replication works

1. **Demand.** `simulate()` draws, from one random generator, each car's inter-arrival time (exponential, rate λ cars/min) and stay (gamma, mean 88 min, shape 1.8). Every strategy sees exactly these cars (common random numbers).
2. **Events.** Before each arrival, departures due by then are processed from a heap of `(departure time, car)` . Each strategy keeps its own occupancy array, so the lots evolve independently.
3. **Choosing a stall.** For each strategy, the arriving car is placed by `assigned()` , `cruising()` or `random_search()` ; a car that finds the lot full leaves (the full-lot driving is recorded). The metres to the stall and back to the exit (the shortest path) are stored per car.
4. **Warm-up.** The first `max(500, 4·λ·mean stay)` cars only fill the lot to steady state and are not measured; then 3,000 cars are measured.
5. **Summary.** `summarize()` averages the metres per parked car and the share of cars turned away; `run_all.py` aggregates 20 replications per setting.

B.3 The arrival rate for a target occupancy

With offered load a = λ·E[S] and c stalls, the Erlang-B probability $B(c, a)$ is the share of cars turned away, and the mean occupancy is $a \cdot (1 - B)/c$. `arrival_rate_for()` solves this for λ by bisection, so each experiment runs at the occupancy it is labelled with (checked by a test to within 0.02).

B.4 Extending the simulation

- **A new strategy:** write a function `(occ, m, rng, p) -> (stall or -1, metres)` in `sim.py` and add it to `STRATEGIES`; it is then run on the same cars as the others.
- **A new lot:** build a `Lot` (grids, entrance, ramps, stall list and search route) in `lot.py`, add it to `build_models()` in `run_all.py`; the tests in `test_sim.py` show how to check its distances against Dijkstra.
- **A new constant:** add a `Param` to `PARAMS` with its source; give it a `sweep` range to include it in the sensitivity analysis.

Appendix C — Simulation parameters, sources and detailed results

Table C1. Simulation constants. Every constant with its value, source and the reason it was chosen. Values without a reliable source are marked *Assumption* and given a range tested in the sensitivity analysis.

Parameter	Value (range)	Source	Basis
Stall width	2.5 m	Israel Ministry of Transport (2000), Ch. D, Table 3 p. 25; Table 9 p. 35	Standard 90° stall width at service level 1 (pairs with a 5.80 m aisle)
Stall length	5 m	Israel Ministry of Transport (2000), Ch. D, Sec. 4.1 p. 11	Minimum 90° stall length: 4.75 m design car + 0.25 m
Aisle width (two-way)	5.8 m	Israel Ministry of Transport (2000), Ch. D, Table 3 p. 25; Sec. 13.3 p. 60	Two-way aisle for 2.50 m stalls at service level 1
Maximum ramp gradient	15%	Israel Ministry of Transport (2000), Ch. D, Sec. 12.5 p. 54	Maximum gradient at service levels 1–2
Ramp transition length	3 m	Israel Ministry of Transport (2000), Ch. D, Sec. 12.5 p. 54	A 3 m transition at half the gradient at each end
Floor-to-floor height	3 m (2.8–3.5)	Assumption	No Israeli figure found; US garages are typically 3.2–3.5 m (Purinton, 2012)
Driving speed inside the lot	10 km/h (5–20)	Israel Ministry of Transport (2000), Ch. D, Sec. 11 p. 45	Design assumption for driving inside a facility (not a measurement)
Fuel use at crawl speed	1 L/h (0.8–2)	Gaines, Rask & Keller (2013), Table 1	Measured idle use 0.279 cc/s = 1.0 L/h; at crawl speed modelled per hour
CO_2 per litre of gasoline	2.35 kg/L	U.S. EPA (2026)	8,887 g CO_2 per gallon
Gasoline price (95, self-service)	₪8.27/L	Israel Ministry of Energy (2026)	October 2026, including VAT
Value of travel time (non-work)	₪27.31/h (25–40)	Nohal PRAT (2021), Table 4.1 p. 118	Parking search counts as travel time (p. 115)
Mean parking duration	88 min (82.5–93.7)	Huang et al. (2020), Table 2 p. 6	Mall garage, gamma-distributed stays
Parking-duration shape (gamma)	1.8 (1.12–2.53)	Huang et al. (2020), Table 2 p. 6	Mid-range of the reported shapes
Headline occupancy	85% (50–98)	Levy, Martens & Benenson (2013), pp. 788–789	Search becomes noticeable above ~85%
Opening hours per day	12 h (10–16)	Assumption	Typical mall hours; only used for yearly totals
Garage floors × aisles × stalls per side	3 × 3 × 12	Assumption	216 stalls, a representative small multi-storey garage
Chance a searching driver notices a free stall (p)	1 (0.5–1)	Assumption	1 = sees every free stall, the most favourable case for the driver

Parameter	Value (range)	Source	Basis
Reservation timeout (prototype)	60 s	Derived	Demo value

Derived values: a step along an aisle = stall width = 2.50 m; aisle to stall = aisle/2 + stall length/2 = 5.40 m; ramp between floors = 2 × 3 m + (3.0 − 0.45)/0.15 = 23.0 m; driving speed = 2.78 m/s.

Additional sources of Table C1: Gaines, L., Rask, E., & Keller, G. (2013). *Which is greener: Idle, or stop and restart?* Argonne National Laboratory. — Israel Ministry of Energy and Infrastructure (2026). *Fuel prices for October 2026*. — Israel Ministry of Transport et al. (2021). *Nohal PRAT 2021: Guidelines for evaluating transport projects*. — Purinton, J. (2012). Mixed-use parking structure design. *The Parking Professional*. — U.S. EPA (2026). *Greenhouse gas emissions from a typical passenger vehicle*. (Full links: `simulation/params.py`, `REFERENCES`.)

Table C2. Garage results by occupancy (p = 1).

Occupancy	Assigned (m/visit)	Searching (m/visit)	Saved (m/car)	Less driving to the stall
50.3%	131.2	181.8	50.5 ± 0.8	43%
60.4%	155.4	216.1	60.6 ± 0.8	44%
70.4%	181.9	252.3	70.4 ± 0.9	44%
80.4%	208.8	289.0	80.2 ± 1.1	43%
85.3%	221.6	306.8	85.2 ± 1.1	43%
90.2%	233.4	323.4	90.0 ± 0.9	44%
95.2%	244.6	339.3	94.6 ± 0.4	44%
98.1%	249.1	345.7	96.6 ± 0.4	44%

Table C3. Saving per car at 85% occupancy, by baseline.

Lot	Compared with	Saved (m/car)	Saved (s/car)
Garage (216 stalls)	Searching alone, p = 1	85.2	30.7
Garage (216 stalls)	Searching alone, p = 0.5	87.7	31.6
Garage (216 stalls)	Random search (upper bound)	741.7	267.0
Demo lot (14 stalls)	Searching alone, p = 1	0.0	0.0
Demo lot (14 stalls)	Searching alone, p = 0.5	11.3	4.1
Demo lot (14 stalls)	Random search (upper bound)	92.6	33.4

Sensitivity. Figure C1 varies each uncertain constant across its range while keeping the others at their base values. The yearly saving is most sensitive to the driving speed inside the garage: between ₪83,000 and ₪332,000 for 20 and 5 km/h around a base of ₪166,000. Floor-to-floor height has no effect, because both drivers use the same ramps.

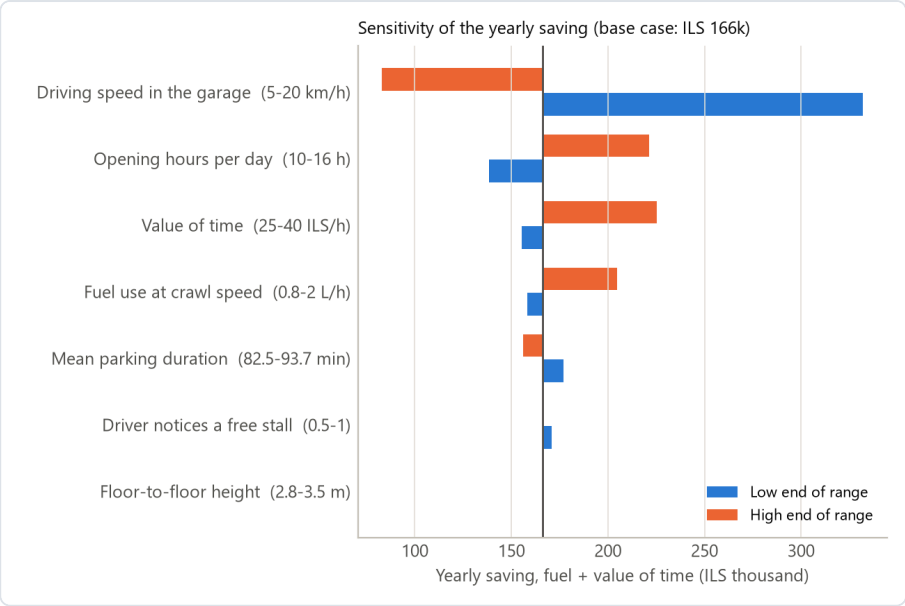


Figure C1. One-at-a-time sensitivity of the yearly saving (fuel plus drivers' time) for the 216-stall garage at 85% occupancy.

Appendix D — Stage-2 documents

The stage-2 documents are attached after this page, in this order, each with its own page numbering:

1. **Programmer manual** (English) — build and toolchain, firmware structure, sensor calibration constants, the road network and A*, the JSON interface and the web page, the automated tests, and a function index.
2. **User manual** (Hebrew) — installing and running the system, the commands, the web page, calibration values and troubleshooting.

The full project documentation (Hebrew), the source code and the tests are in the repository: github.com/AseelKurdy/smart-parking-lot.

Smart Parking System — Programmer Manual

Version	`smart_parking_v9.ino` + `parking_page.h` (measured 1:100 model, October 2026)
Target	ESP32-WROOM32 (CH9102 USB bridge), Arduino framework, ESP32 core 3.0.1
Date	24/10/2026
Authors	Aseel Kurdy, Nayef Habiballah, Osama Abdulhalim
Instructors	Itai Dabran, Ido Ram, Diana Cohen
Repository	github.com/AseelKurdy/smart-parking-lot — firmware in `firmware/`, tests in `tests/`, simulation in `simulation/`, documents in `docs/`

> The code comments are in Hebrew; this manual is in English so that code identifiers stay readable.

1. Build & toolchain

IDE	Arduino IDE 2.x
Board package	`esp32` by Espressif (3.0.1)
Board selection	**ESP32 Dev Module**
Serial baud	9600
Libraries	`WiFi.h`, `WebServer.h` — both ship with the ESP32 core

1.1 File layout

```
firmware/   (in the GitHub repository; locally the two sketch folders sit at the project root)
smart_parking_v9/
├── smart_parking_v9.ino     <- firmware
└── parking_page.h           <- web page (HTML/CSS/JS) as a PROGMEM string
smart_parking_v9_research/   <- identical, but distances are reported as real-scale metres
tests/                       <- simulator, test suite, fuzz, ESP32 compile check (section 13)
docs/                        <- builds the PDF manuals
```

`parking_page.h` **must** sit in the sketch folder; it is included only when `ENABLE_WEB_VIEW` is 1.

1.2 Sketch sections

Section	Content
`[1]`	Pins, sensor calibration, timing
`[1ב]`	Web-view configuration
`[2]`	Measured dimensions of the model
`[3]`	Spot table and car bookkeeping
`[4]`	Road network and A*
`[4ב]`	Spot state, text picture, JSON builder
`[5]`	Lot operations (entry, park, leave, reroute, timeout, sensor events)
`[6]`	Command parser
`[7]`	Sensor reading and entrance classification
`[7ב]`	HTTP server
`[8]`	`setup()` / `loop()`

2. Configuration constants

All tunables are at the top of the sketch. Changing them requires a re-upload.

2.1 Hardware pins — section [1]

```
const int trigPin1 = 27, echoPin1 = 14;   // entrance sensor (side-facing)
const int trigPin2 = 26, echoPin2 = 25;   // spot-0 sensor (top-down)
const int ledPin   = 2;                   // on-board blue LED
```

2.2 Sensor calibration — section [1]

```
// entrance sensor: what it reads on the model
const float GATE_EMPTY_CM     = 30.0;  // empty road = the wall opposite (no echo = empty too)
const float GATE_ENTRY_CM     = 22.0;  // a car entering (measured)
const float GATE_CAR_BELOW_CM = 25.0;  // detection threshold: a reading below this is a car
const float GATE_EXIT_CM      = 0.0;   // optional separate exit lane seen by the same sensor; 0 = none
const int   GATE_CONFIRM_SAMPLES = 2;
const int   GATE_CLEAR_SAMPLES   = 2;

// spot 0 (top-down): empty 25-26 cm; car 2.5 cm high 22.5-23.5 cm, car 1.5 cm high 23.5-24.5 cm
const float SPOT_ARRIVE_CM = 24.7;  // below -> a car arrives
const float SPOT_LEAVE_CM  = 24.9;  // above -> the car left
const float MIN_VALID_CM   = 2.0;   // shorter readings are electrical noise
```

- **Entrance:** a reading below `GATE_CAR_BELOW_CM` (**25.0 cm**) is a car, anything at or above it (or no echo) is the empty road. `GATE_EMPTY_CM` and `GATE_ENTRY_CM` record the measured readings for reference and for the boot banner. With an exit lane (`GATE_EXIT_CM > 0`) a car reading is assigned to whichever of `GATE_ENTRY_CM` / `GATE_EXIT_CM` is nearer. `printGateBands()` prints the threshold at boot.
- **Spot 0:** two thresholds (hysteresis). They must satisfy `car readings < SPOT_ARRIVE_CM < SPOT_LEAVE_CM < empty readings`. The values are chosen so that a low car (1.5 cm high, 23.5–24.5 cm) is also detected; between 24.5 (low car) and 25.0 (empty stall) this leaves only about 0.2 cm below and 0.1 cm above the band — at the limit of the HC-SR04's resolution. Getting the order wrong latches the sensor in one state (a real bug in an early version).
- **Optional exit lane:** with `GATE_EXIT_CM > 0` (e.g. exit 22, entry 26, `GATE_CAR_BELOW_CM` 28), a car in the exit lane removes a car from the lot (`onGateExit()`), see section 6.3.

2.3 Timing — section [1]

Constant	Value	Effect
`SAMPLE_INTERVAL_MS`	500	Sensor sampling period
`SPOT_CONFIRM_SAMPLES`	4	Agreeing samples for a spot-0 change (≈ 2 s)
`GATE_CONFIRM_SAMPLES` / `GATE_CLEAR_SAMPLES`	2 / 2	Entrance: samples to count a car / to re-arm (≈ 1.1 s)
`CROSSTALK_DELAY_MS`	60	Gap between the two sensor reads (HC-SR04 datasheet)
`CHECK_DEFAULT_S`	3	Default duration of `check`
`BLINK_INTERVAL_MS`	300	LED blink half-period
`RESERVATION_TIMEOUT_MS`	60000	Reservation lifetime
`EXIT_MATCH_MS`	20000	Exit lane only: a car that just left spot 0 is the one seen in the exit lane

2.4 Web view — section `[1ב]`

```
#define ENABLE_WEB_VIEW 1          // 0 = no WiFi; the Serial Monitor draws the text picture instead
const char* WEB_SSID = "SmartParking";
const bool  TEXT_MAP = false;      // true = also draw the text picture while the page runs
```

`showTextMap()` returns `TEXT_MAP || !webRunning`, so the text picture is an automatic fallback.

2.5 Measured model — section `[2]`

```
const float ENT_TO_FLOOR0_CM   = 22.0;  // entrance -> floor-0 lane, just before spot 0
const float ENT_TO_RAMP_CM     = 17.5;  // entrance -> start of the ramp
const float RAMP_CM            = 20.0;  // ramp start -> floor-1 entrance
const float FLOOR1_TO_FIRST_CM = 7.5;   // floor-1 entrance -> floor-1 lane, just before spot 7
const float SPOT_WIDTH_CM      = 2.5;   // lane step between neighbouring stalls
const float SPOT_LENGTH_CM     = 5.0;   // lane -> inside the stall
const float CAR_LENGTH_CM = 4.0, CAR_WIDTH_CM = 2.0, CAR_HEIGHT_CM = 2.5;
const char* DIST_UNIT = "cm";           // "m" in the research variant (1:100)
const int   FLOORS = 2, SPOTS_PER_FLOOR = 7;
```

Resulting distances: floor 0 = 27.0, 29.5 … 42.0 cm; floor 1 = 50.0, 52.5 … 65.0 cm. Every constant is reported in the JSON (`geo`) so the page shows the same numbers.

---

3. The lot model

3.1 Why a road network

On the measured model the ramp starts *before* the floor-0 stalls (17.5 cm vs 22 cm) and the first road segment is much longer than the stall pitch. The earlier fixed-step grid cannot represent this, so the lot is a small graph whose edges carry the measured lengths.

3.2 Nodes and edges — section `[4]`

Node index	Meaning
`NODE_ENT` (0)	main entrance
`NODE_RAMP` (1)	start of the ramp
`NODE_FLOOR1` (2)	floor-1 entrance (top of the ramp)
`NODE_LANE0 + i` (3–16)	point in the lane in front of spot `i`
`NODE_SPOT0 + i` (17–30)	inside spot `i`

`buildRoadNetwork()` creates the edges:

```
addEdge(NODE_ENT,    NODE_LANE0,                  ENT_TO_FLOOR0_CM);
addEdge(NODE_ENT,    NODE_RAMP,                   ENT_TO_RAMP_CM);
addEdge(NODE_RAMP,   NODE_FLOOR1,                 RAMP_CM);
addEdge(NODE_FLOOR1, NODE_LANE0 + SPOTS_PER_FLOOR, FLOOR1_TO_FIRST_CM);
for each spot i:
  if i is not the last on its floor: addEdge(NODE_LANE0+i, NODE_LANE0+i+1, SPOT_WIDTH_CM);
  addEdge(NODE_LANE0+i, NODE_SPOT0+i, SPOT_LENGTH_CM);
```

30 edges, 31 nodes. Edges are undirected.

3.3 Spot table — section `[3]`

```
const int TOTAL_SPOTS = 14;
const uint8_t SPOT_FL[TOTAL_SPOTS] = {0,0,0,0,0,0,0, 1,1,1,1,1,1,1};
int spotIndexOnFloor(int i) { return i % SPOTS_PER_FLOOR; }   // 0 = first from the floor's entrance
```

Spots are numbered by increasing distance from the entrance (spot 0 nearest). Parallel arrays rather than a `struct` : **the Arduino IDE inserts auto-generated prototypes before user types are declared**, so a `struct` in a function signature fails to compile.

---

4. State variables

4.1 Per-spot arrays — section `[3]`

Variable	Type	Meaning
`spotDist[14]`	`float`	A* distance from the entrance (cm), computed once in `setup()`
`occupied[14]`	`bool`	A car is physically parked here
`assignedCarId[14]`	`int`	`>= 0` → reserved for that car; `-1` → not reserved
`reservedAtMs[14]`	`unsigned long`	`millis()` when the reservation was created
`parkedCarId[14]`	`int`	Which car is parked here
`wasCorrect[14]`	`bool`	Whether the parked car was in its assigned spot

4.2 Global state

Variable	Meaning
`nextCarId`	Next ID to hand out; the first car gets **0**
`carExited[500]`	Whether car `i` has left the lot
`spotSensedStable` , `spotDisagreeCount`	Debounced spot-0 state and the run of contradicting samples
`gateLatched` , `gateCand` , `gateCandCount` , `gateClearCount`	Entrance state machine (section 8.3)
`spotLeavePending` , `spotLeaveAtMs`	Exit lane only: the car that just left spot 0
`firstSample`	Suppresses events on the very first sample
`edgeA/edgeB/edgeLen/edgeCount`	The road network
`statusSeq`	Incremented by `status` ; the browser watches it
`webRunning`	True only after the AP and the server started

Memory: under 3 KB in total, negligible against the ESP32's ~320 KB of SRAM.

5. A* — section `[4]`

5.1 `float aStarDistance(int goalSpot)`

Shortest driving distance (cm) from `NODE_ENT` to `NODE_SPOT0 + goalSpot`, or `-1`. The open set is a linear scan over the 31 nodes; neighbours come from a scan over the 30 edges. A spot node other than the goal is never expanded — **a car cannot drive through a stall**.

5.2 Heuristic

```
float heuristic(int node, int goalSpot) {
  if (node == NODE_SPOT0 + goalSpot) return 0;
  if (node is a lane node on the goal's floor)
    return |index(node) - index(goal)| * SPOT_WIDTH_CM + SPOT_LENGTH_CM;
  return SPOT_LENGTH_CM;
}
```

Admissible: every route ends with the `SPOT_LENGTH_CM` edge into the stall, and from a lane node on the same floor the car must also drive along the lane to the stall. h never overestimates, so A* returns the shortest path. (No closed set; a node can be re-opened. At 31 nodes this costs nothing.)

5.3 `void computeSpotDistances()`

Called once from `setup()`: builds the network and runs A* for all 14 spots into `spotDist[]`. **No search happens at runtime.** A single Dijkstra from the entrance would give every distance in one pass; A* was the specified algorithm.

---

6. Lot logic — section `[5]`

6.1 Query helpers

Function	Returns
`int findAvailableSpot(float* outDist)`	Nearest spot not occupied, not reserved and not blocked; `-1` if none
`int whereIsCarParked(int carId)` / `whereIsCarReserved(int carId)`	Spot index, else `-1`
`bool isCarInLot(int carId)`	Car exists and has not exited
`int newestUnparkedCar()`	Highest-numbered car still in the lot that has not parked
`bool spot0Blocked()`	Sensor sees an object at spot 0 but no car is registered there
`int collectWaitingCars(int* ids, int maxIds)`	Cars in the lot that have not parked (total; first `maxIds` stored)

6.2 Operations

- `int assignNewCar()` — refuses with `LOT FULL` before allocating an ID (a full lot does not burn IDs), registers the car, reserves the nearest spot, prints `[ENTRY]` and the next-car recommendation.
- `void parkCarAt(int spot, int carId)` — validates (spot range, id, car in lot, not already parked, spot not occupied), then parks. Parking in a spot reserved for another car is **accepted** (a barrier cannot stop a car); the displaced car is rerouted, or leaves if the lot is full.
- `void leaveCar(int carId, bool fromSensor)` — the car leaves the lot, parked or still driving.
- `void checkReservationTimeouts()` — every `loop()`: releases reservations older than 60 s; the car stays in the lot.

6.3 Sensor events

Function	When	What
`onSpotSensorArrive(cm)`	spot 0 confirmed occupied	parks the car the spot is reserved for, else `newestUnparkedCar()`, else marks spot 0 `BLOCKED`
`onSpotSensorLeave(cm)`	spot 0 confirmed empty	the car registered there leaves the lot
`assignNewCar()`	entrance: car confirmed	same as the `new` command
`onGateExit(cm)`	exit lane only (`GATE_EXIT_CM > 0`)	removes a car: the one that just left spot 0 is matched (`EXIT_MATCH_MS`), else `pickCarForExit()` = car at spot 0, else lowest parked id, else any car in the lot

---

7. Spot state — section `[4ב]`

```
const int SP_FREE = 0, SP_RESERVED = 1, SP_OCCUPIED = 2, SP_BLOCKED = 3;
int spotState(int i);   // occupied > reserved > blocked (spot 0 only) > free
```

---

8. Sensor layer — section `[7]`

8.1 `float readDistanceCm(int trigPin, int echoPin)`

10 µs trigger, `pulseIn()` with a 30 ms timeout; returns `-1` when no echo returns. `(duration / 2.0) / 29.1` converts µs to cm.

8.2 `bool validReading(float cm)`

`cm >= MIN_VALID_CM`. Shorter pulses are noise on the Echo line (measured 0.0–0.1 cm).

8.3 Entrance — `int classifyGate(float cm)` and the state machine in `loop()`

`classifyGate()` returns `GATE_EMPTY` for no echo or any reading at or above `GATE_CAR_BELOW_CM`; below it, `GATE_ENTRY`, or `GATE_EXIT` when an exit lane is configured and its reading is the nearer one; noise (< 2 cm) is

ignored entirely. A car is counted after `GATE_CONFIRM_SAMPLES` equal readings in a row and only once (`gateLatched`) until the road reads empty `GATE_CLEAR_SAMPLES` times.

8.4 Spot 0 — debounce in `loop()`

```
bool spotSensed = spotValid && distSpot < (spotSensedStable ? SPOT_LEAVE_CM : SPOT_ARRIVE_CM);
if (!spotValid || spotSensed == spotSensedStable) spotDisagreeCount = 0;
else if (++spotDisagreeCount >= SPOT_CONFIRM_SAMPLES) { /* change state, fire the event */ }
```

The active threshold depends on the current state (hysteresis); an invalid reading resets the count, because noise arrives in bursts.

---

9. Command parser — section `[6]`

`Serial -> lineBuf (48) -> handleCommand() -> splitTokens() -> parseNumber() -> operation`

Command	Args	Handler
`new`	0	`assignNewCar()`
`park`	2	`parkCarAt(spot, id)`
`leave` / `cancel`	1	`leaveCar(id, false)`
`status`	0	`printStatus()`
`check`	0–1	sets `checkUntilMs`
`help`	0	`printHelp()`

Commands are case-insensitive (`strcasecmp`), argument counts are exact, `parseNumber()` (`strtol`) rejects trailing junk and huge values, and a token longer than 15 characters is stored empty instead of being truncated into a different valid value.

---

10. Web layer

10.1 Server — section `[7ב]`

```
WebServer webServer(80);
webServer.on("/",      handleWebRoot);    // PARKING_PAGE from PROGMEM
webServer.on("/state", handleWebState);   // buildStateJson()
```

`startWebView()` starts an open soft AP named `SmartParking` (no internet). On failure `webRunning` stays false and the Serial Monitor draws the text picture.

10.2 `String buildStateJson()` — the payload

```
{"up":1,"seq":0,"ttl":60,"floors":2,"per":7,"unit":"cm",
 "geo":{"e0":22.0,"er":17.5,"ramp":20.0,"f1":7.5,"sw":2.5,"sl":5.0,"car":[4.0,2.0,2.5]},
 "real":0,"next":0,"nextd":27.0,
 "sp":[{"f":0,"k":0,"s":0,"car":-1,"ok":1,"left":0,"d":27.0}, ...],
 "waitN":0,"wait":[]}
```

Key	Meaning
`up`, `seq`, `ttl`	uptime (s), `status` counter, reservation lifetime (s)
`floors`, `per`, `unit`	2 floors, 7 spots per floor, distance unit
`geo`	measured lengths: entrance→floor-0 lane, entrance→ramp, ramp, floor-1→lane, stall width/length, car L/W/H
`real`	index of the physical spot
`next`, `nextd`	spot for the next car (`-1` = full) and its distance
`sp[i]`	`f` floor, `k` position on its floor, `s` state, `car`, `ok`, `left` seconds, `d` distance
`waitN`, `wait[k]`	cars still driving (total) and up to 16 of them: `car`, `spot` (`-1` = expired), `left`

Size: about 1.5 KB in the worst case.

10.3 Browser side — `parking_page.h`

One PROGMEM raw string with HTML, CSS and JavaScript. **No CDN and no external fonts** — the AP has no internet.

The model is drawn in centimetres (`viewBox="0 0 58 48"`), traced from the photo of the model (`M` holds where things are). Stall and car sizes come from `geo`, the distances shown are the measured ones from the board. Three stacked SVG layers — base (floor 0, road, entrance, sensors), ramp, and deck (floor 1 with the U-turn) — plus an overlay for routes; in 3D the board is tilted with CSS, the deck layer is lifted (`translateZ`) and the ramp layer tilted so it meets it.

Function	Role
`poll()`	`fetch('/state')` every second, 2.5 s abort
`draw(s)`	counters, banner, side cards; redraws stalls/routes when the state changes; starts arrival animations
`buildBoard()`	draws the model once (grass, roads, floors, ramp, U-turn, deck, sensors)
`stall(p)`, `carAt(r)`, `route(i)`	where a stall is, where a car stands in it, the driving path to it
`drawSpots(s)`	state of every stall (free / next / reserved / parked right or wrong / blocked)
`drawRoutes(s)`	dashed route of the next car, orange route of the selected stall
`animateArrival(car, spot)`, `stepMovers()`	a new car drives from the entrance to its stall
`drawWaiting`, `drawInfo`, `drawDimsTable`, `drawDims`	side cards and the dimension labels
`setView(v)`	2D / 3D (`?view=3d` opens in 3D)

11. Main loop — section `[8]`

```
void loop() {
  if (webRunning) webServer.handleClient();   // 1. HTTP
  while (Serial.available()) { ... }          // 2. commands
  checkReservationTimeouts();                 // 3. expiries
  if (now - lastSampleTime >= SAMPLE_INTERVAL_MS) { ... }   // 4. entrance + spot 0 sensors
  // 5. LED: off = no car, solid = right car, blinking = wrong car
}
```

Blocking: `CROSSTALK_DELAY_MS` (60 ms) and up to 30 ms per `pulseIn()`. Printing the text picture at 9600 baud blocks for about 1.4 s, which is why it is only a fallback.

12. How to modify common things

1. **Distances:** edit the constants of section `[2]`. A*, the JSON, the page labels and the tests follow automatically.
2. **Sensor calibration:** run `check 10` empty and with a car; put the readings in `GATE_EMPTY_CM` / `GATE_ENTRY_CM`; for spot 0 keep `car < SPOT_ARRIVE_CM < SPOT_LEAVE_CM < empty`.
3. **Exit lane:** set `GATE_EXIT_CM` to the exit-lane reading (and `GATE_ENTRY_CM` to the entry-lane reading).
4. **Physical spot:** change `REAL_SPOT_INDEX`; sensor handlers, LED and `[PHYSICAL]` follow.
5. **Number of spots / layout:** `TOTAL_SPOTS`, `SPOTS_PER_FLOOR`, `SPOT_FL`, `buildRoadNetwork()`, and the drawing positions in `M` / `stall()` in the page.
6. **Disable WiFi:** `#define ENABLE_WEB_VIEW 0`.

7. **RFID instead of the ID counter:** replace the attribution in `onSpotSensorArrive()` with a UID lookup; `parkCarAt()` already takes an explicit `carId`.

13. Testing (folder `tests/`)

Tool	What it does
`host_sim/`	The **unmodified** `.ino` compiled on the PC with stand-ins for Serial, the clock, both HC-SR04 sensors (with noise models), the LED and the web handlers. `build.ps1` builds variants: text picture, as shipped, WiFi off, exit lane.
`run_tests.ps1 -Sim`	The 38 original hardware tests plus `SIM-*` (sensors, LED) and `VIS-*` (JSON, picture). `-Slow`, `-Soak`, `-Only`. With `-Port COMx` it drives the real board (set `TEXT_MAP = true` first).
`fuzz_model.py`	Independent Python model of the rules; after every random operation the firmware's JSON must equal the model.
`edge_cases.py`	3-digit car IDs, more than 16 waiting cars, worst-case payload size.
`esp32_syntax_check.ps1`	Compiles the sketch with the real Xtensa compiler and ESP32 core headers (syntax/type check, no link).

14. Pitfalls

Pitfall	Why it matters
`struct` in a function signature	Arduino's auto-prototypes come before the type → compile error
Spot thresholds in the wrong order	The sensor latches and never fires a leave event
The wall at the entrance closer than `GATE_CAR_BELOW_CM`	Every reading counts as a car (ghost cars)
`atoi` on user input	Maps `"abc"` to 0; use `parseNumber()`
Hebrew in Serial output	The Serial Monitor has no RTL support; all runtime output is English
`millis()` rollover (~49 days)	Subtraction-based comparisons are safe; `checkUntilMs` uses a direct comparison
Removing `CROSSTALK_DELAY_MS`	One sensor picks up the other's echo

15. Function index — `smart_parking_v9.ino`

Function	Section	Purpose
`spotIndexOnFloor`	`[3]`	Position of a spot on its floor
`addEdge`, `buildRoadNetwork`	`[4]`	Build the road network
`heuristic`, `aStarDistance`, `computeSpotDistances`	`[4]`	A* and the distance table
`spot0Blocked`, `findAvailableSpot`, `whereIsCarParked`, `whereIsCarReserved`, `clearReservationFor`, `isCarInLot`, `newestUnparkedCar`	`[4]`	Queries
`spotState`, `reservationSecondsLeft`, `collectWaitingCars`	`[4↵]`	State
`printSpotCell`, `printMapRow`, `printMapLine`, `printWaitingLine`, `printLotMap`	`[4↵]`	Text picture
`buildStateJson`	`[4↵]`	JSON for the page
`registerCar`, `reserveSpotFor`, `printSpotAndDist`, `printNextRecommendation`, `printLotUpdate`	`[5]`	Helpers
`assignNewCar`, `rerouteCar`, `parkCarAt`, `leaveCar`, `checkReservationTimeouts`	`[5]`	Operations
`onSpotSensorArrive`, `onSpotSensorLeave`, `pickCarForExit`, `onGateExit`	`[5]`	Sensor events
`printStatus`, `printHelp`	`[5]`	Output
`splitTokens`, `parseNumber`, `argsOk`, `handleCommand`	`[6]`	Parser
`readDistanceCm`, `validReading`, `classifyGate`, `printGateBands`	`[7]`	Sensors
`handleWebRoot`, `handleWebState`, `startWebView`	`[7↵]`	HTTP

חניון חכם — מדריך למשתמש

גרסת תוכנה	`smart_parking_v9.ino` (דגם שנמדד)
תאריך	24/10/2026
מגישים	Aseel Kurdy, Nayef Habiballah, Osama Abdulhalim
מנחים	Itai Dabran, Ido Ram, Diana Cohen
קוד המקור	github.com/AseelKurdy/smart-parking-lot (הקושחה בתיקייה `firmware/smart_parking_v9`)

1. מה המערכת עושה

המערכת מנהלת דגם של חניון בן **שתי קומות ו-14 חניות** (קנה מידה 1:100):

- כשרכב נכנס, המערכת מקצה לו אוטומטית את **החניה הפנויה הקרובה ביותר לכניסה** ומודיעה לאן לנסוע ובאיזה מרחק.
- כשהרכב חונה, המערכת בודקת אם זו אכן החניה שהוקצתה לו.
- אם כן — הנורית הכחולה שעל הלוח **דולקת באור קבוע**.
- אם לא — הנורית **מהבהבת** כאתראה.

חניה מספר 0 היא החניה האמיתית, עם חיישן מעליה ונורית. 13 החניות הנותרות וירטואליות ונשלטות בפקודות. חיישן נוסף בכניסה מזהה כל רכב שנכנס.



הדגם במבט מלמעלה

2. הכנה ראשונה

2.1 מה צריך

- מחשב עם Arduino IDE מותקן
- הלוח ESP32 מחובר בכבל USB
- שני קבצי הפרויקט: `smart_parking_v9.ino` ו- `parking_page.h`
- טלפון או מחשב נוסף עם WiFi — לצפייה בתמונת החניון

2.2 התקנה חד-פעמית

שלב 1 — הוספת תמיכה בלוח ESP32

1. בסרגל העליון: **File** ← **Preferences**
2. בשדה **Additional Boards Manager URLs** הדביקו:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

3. לחצו **OK**
4. **Tools** ← **Board** ← **Boards Manager**
5. חפשו `esp32` והתקינו את החבילה של **Espressif**

שלב 2 — דרייבר ה-USB

הלוח משתמש בשבב **CH9102**. אם הפורט לא מופיע ברשימה, יש להתקין את הדרייבר של CH9102.

2.3 העלאת התוכנה

1. ודאו ששני הקבצים — `smart_parking_v9.ino` ו-`parking_page.h` — נמצאים **באותה תיקייה** (`smart_parking_v9`).
2. פתחו את `smart_parking_v9.ino` ב-Arduino IDE.
3. **Tools** ← **Board** ← **ESP32 Dev Module**
4. **Tools** ← **Port** ← הפורט של הלוח (למשל `COM10`)
5. לחצו על כפתור החץ (**Upload**)
6. המתינו ל-**Done uploading**

> אם ההעלאה נתקעת על "Connecting..." — החזיקו את כפתור **BOOT** שעל הלוח כשההעלאה מתחילה.

---

3. הפעלה

3.1 פתיחת ה-Serial Monitor

1. **Tools** ← **Serial Monitor**
2. קצב: **baud 9600**
3. סוף שורה: **New Line**

3.2 הודעות הפתיחה

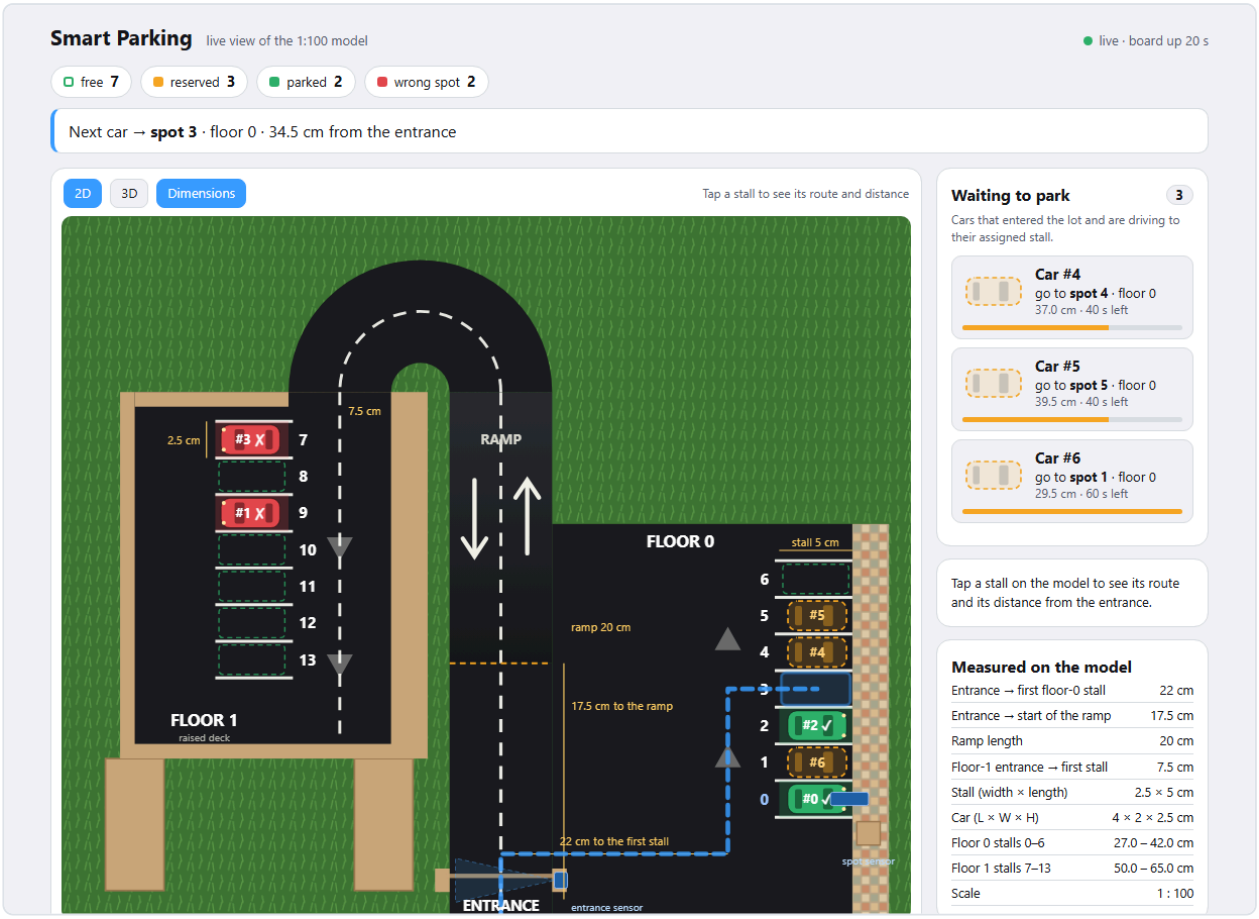
```
[BOOT]   ready: 14 spots, 2 floors
[BOOT]   nearest spot 27.0 cm | farthest 65.0 cm
[BOOT]   reservations expire after 60 s
[BOOT]   entrance sensor: empty ~30.0 cm, car ~22.0 cm -> a car is counted below 25.0 cm
[BOOT]   spot 0 sensor: car arrives below 24.7 cm, leaves above 24.9 cm
[WEB]    picture page: join WiFi "SmartParking" then open http://192.168.4.1
[HELP]   new | park <spot> <id> | leave <id> (alias: cancel) | status | check [sec]
[NEXT]   next car -> spot 0 (floor 0)  dist 27.0 cm
```

אם השורות האלה מופיעות — המערכת מוכנה. כל המרחקים הם בס"מ של הדגם (בקנה מידה 1:100 — אותם מספרים במטרים בחניון אמיתי).

---

4. התמונה הגרפית של החניון

זהו החלק המומלץ להדגמה: ציור חי של הדגם במבט מלמעלה, כמו בתמונה שלו — הכביש הראשי והכניסה, קומה 0 מימין, הרמפה וה-U, וקומה 1 על המשטח המוגבה משמאל — עם החניות והרכבים במידות האמיתיות.



תמונת החניון בעמוד האינטרנט

4.1 איך נכנסים

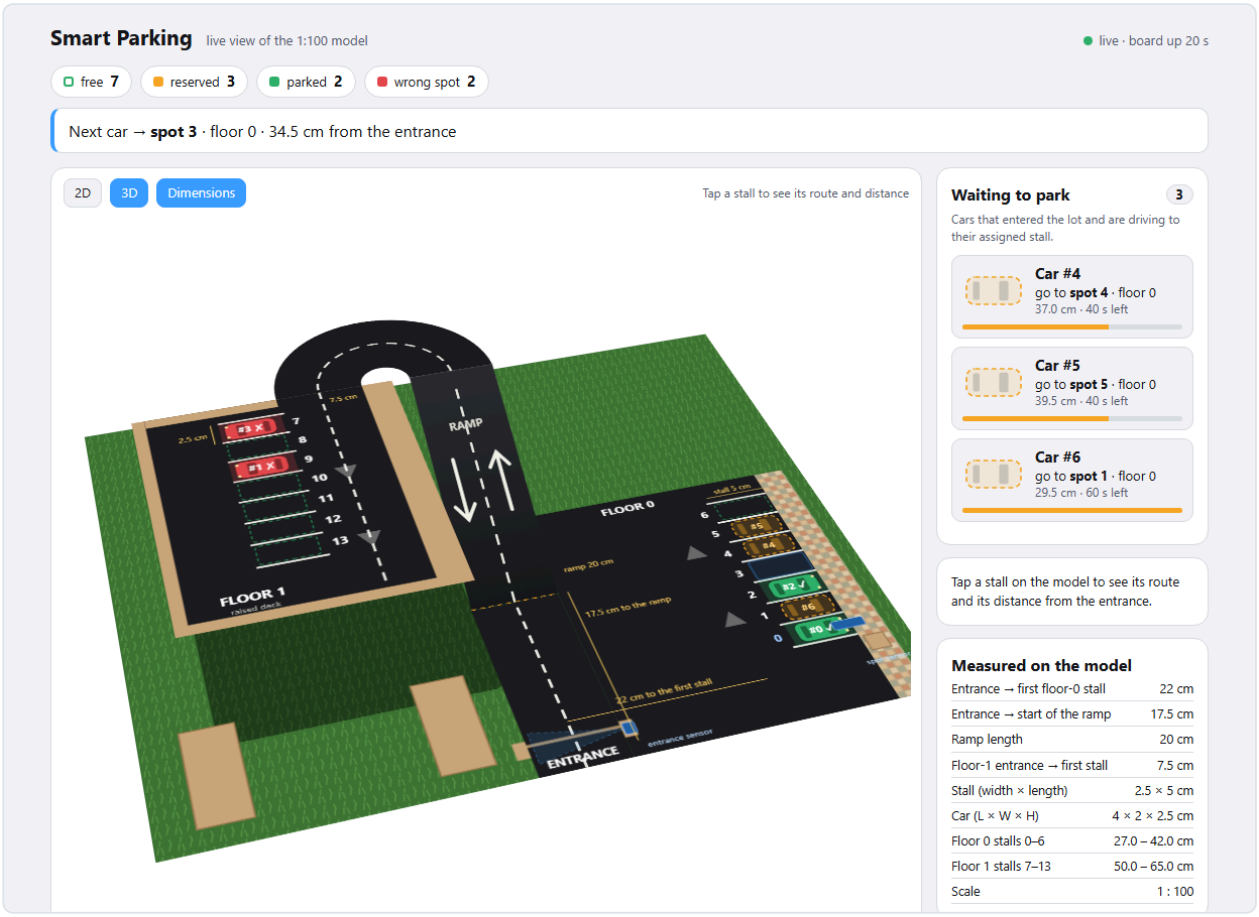
1. בטלפון או במחשב, פתחו את רשימת רשתות ה-WiFi
2. התחברו לרשת `SmartParking` (ללא סיסמה)
3. פתחו דפדפן וגשו לכתובת `http://192.168.4.1` (הכתובת מודפסת בשורת `[WEB]`)

> הרשת אינה מחוברת לאינטרנט. הטלפון עשוי להתריע על כך — זה תקין; אשרו "להישאר מחובר".

4.2 מה רואים בעמוד

אזור	תוכן
סרגל עליון	מחוון "live" ומשך הפעילות של הלוח
שורת ספירה	כמה חניות פנויות / משוריינות / תפוסות / שגויות / חסומות
פס כחול	לאן יישלח הרכב הבא, ובאיזה מרחק
הדגם	שתי הקומות; קו כחול מקווקו = המסלול של הרכב הבא; רכב חדש "נוסע" מהכניסה אל החניה שלו
כפתורים	**2D** / **3D** (קומה 1 מורמת מעל קומה 0) / **Dimensions** (המידות שנמדדו על הציור)
בצד	הרכבים שנכנסו וטרם חנו (עם טיימר השריון), פרטי החניה שנבחרה, וטבלת המידות שנמדדו

לחיצה על חניה מציגה את המסלול אליה (בכתום), את המרחק שלה מהכניסה ואת הפירוק שלו — למשל לחניה 12: 17.5 (עד הרמפה) + 20 (רמפה) + 7.5 + 5 × 2.5 + 5 (לתוך החניה) = 62.5 ס"מ.



תצוגת 3D

אפשר לפתוח ישר בתצוגת 3D בכתובת `http://192.168.4.1/?view=3d`.

4.3 מקרא הצבעים

צבע	משמעות
מסגרת ירוקה מקווקוות	חניה פנויה
כחול מהבהב	לכאן יישלח הרכב הבא
כתום (רכב מקווקו)	משוריינת — רכב בדרך
רכב ירוק עם ✓	רכב חונה בחניה **הנכונה**
רכב אדום עם ✗	רכב חונה בחניה **השגויה**
אפור עם ⨯	חסומה — החיישן רואה עצם ללא רכב רשום

העמוד מתעדכן מעצמו כל שנייה. הקלדת `status` ב-Serial Monitor גורמת לו להבהב ולהתרענן מיד.

---

5. הפקודות

הפקודות נכתבות בשורת ההודעה שבראש ה-Serial Monitor ונשלחות ב-**Enter**. **אין הבדל בין אותיות גדולות לקטנות.**

5.1 טבלת הפקודות

פקודה	מה היא עושה
`new`	רכב חדש נכנס (כמו רכב שעובר מול חיישן הכניסה)
`park <spot> <id>`	הרכב `<id>` חונה בחניה `<spot>`
`leave <id>`	הרכב `<id>` יוצא מהחניון — בין אם חנה ובין אם היה בדרך
`cancel <id>`	זהה ל-`leave`
`status`	שורת סיכום, ורענון מיידי של התמונה בעמוד
`check [sec]`	מרחקי החיישנים. ברירת מחדל 3 שניות, טווח 1–60
`help`	רשימת הפקודות

5.2 דוגמאות

```
new             ->  a new car enters
park 3 1        ->  car #1 parks in spot 3
leave 1         ->  car #1 leaves the lot
check 10        ->  sensor distances for 10 seconds
status          ->  summary + refresh the picture
```

---

6. איך לקרוא את הפלט

תג	משמעות
`[BOOT]`	הודעות הפעלה
`[WEB]`	פרטי החיבור לעמוד הגרפי
`[SENSOR]`	חיישן זיהה שינוי
`[ENTRY]`	רכב נכנס וקיבל הקצאה (מספר החניה, הקומה והמרחק)
`[PARK]`	רכב חנה — `CORRECT` או `WRONG SPOT!`
`[LEAVE]`	רכב יצא מהחניון
`[REROUTE]`	רכב שחניתו נתפסה קיבל חניה חדשה
`[TIMEOUT]`	שריון פג
`[NEXT]`	לאן יישלח הרכב הבא
`[STATUS]` / `[VISUAL]`	שורת סיכום / העמוד רוענן
`[CHECK]`	מרחקי החיישנים
`[WARN]` / `[ERROR]`	אזהרה / שגיאה בפקודה

6.1 דוגמה לרצף מלא

```
[SENSOR] entrance: car detected at 22.0 cm
[ENTRY]  car #0  -> go to spot 0 (floor 0)  dist 27.0 cm  [PHYSICAL]
[NEXT]   next car -> spot 1 (floor 0)  dist 29.5 cm
[SENSOR] spot 0: object detected at 23.0 cm
[PARK]   car #0  @ spot 0  dist 27.0 cm  -> CORRECT
[NEXT]   next car -> spot 1 (floor 0)  dist 29.5 cm
```

רכב עבר מול חיישן הכניסה, הופנה לחניה 0, הגיע אליה, והחניה נכונה ← הנורית דולקת באור קבוע.

---

7. תרחישי הדגמה

7.1 חניה נכונה — נורית קבועה

צעד	פעולה	מה קורה
1	העבירו רכב מול חיישן הכניסה (או הקלידו `new`)	רכב #0 מופנה לחניה 0; בעמוד הוא "נוסע" אליה
2	העמידו את הרכב בחניה 0, מתחת לחיישן	לאחר כ-2 שניות: `[PARK] ... CORRECT`
3	הסתכלו על הנורית	**דולקת באור קבוע**; בעמוד הרכב ירוק
4	הוציאו את הרכב	`[LEAVE] car #0 ... (sensor)` והנורית כבית

7.2 חניה שגויה — נורית מהבהבת

צעד	פעולה	מה קורה
1	`new`	רכב #0 ← חניה 0
2	`new`	רכב #1 ← חניה 1
3	`park 7 0`	רכב #0 חונה בחניה 7 ← `WRONG SPOT!`, חניה 0 מתפנה
4	העמידו רכב בחניה 0	רכב #1 (שהיה בדרך לחניה 1) נרשם בחניה 0 ← `WRONG SPOT!` והנורית **מהבהבת**

7.3 ניתוב מחדש

צעד	פעולה	מה קורה
1	`new`, `new`	רכב #0 ← חניה 0, רכב #1 ← חניה 1
2	`park 1 0`	רכב #0 משתלט על חניה 1
3	—	`[REROUTE] car #1: spot 1 was taken, go to spot 0`

7.4 חניון מלא

הקלידו `new` ארבע-עשרה פעמים. בפעם ה-15 יופיע `[ENTRY] LOT FULL - no car created`.

7.5 פקיעת שריון

הקלידו `new` והמתינו דקה בלי לחנות: `[TIMEOUT] car #0 did not reach spot 0 within 60 s - spot released`. הרכב נשאר בחניון (ובעמוד, ברשימת הממתינים) עד `leave`.

---

8. בדיקת החיישנים

הקלידו `check` (או `check 10`):

```
[CHECK]  entrance 30.0 cm | spot 25.6 cm | spot 0 free | gate empty
[CHECK]  entrance 22.1 cm | spot 25.6 cm | spot 0 free | gate CAR (entry)
```

8.1 ערכים תקינים (נמדדו על הדגם)

מצב	חיישן כניסה	חיישן חניה 0
ללא רכב	~30 ס"מ (הקיר)	25–26 ס"מ
עם רכב	~22 ס"מ	22.5–23.5 ס"מ (רכב נמוך, 1.5 ס"מ: 23.5–24.5)
הסף בקוד	רכב מתחת ל-25.0	מגיע מתחת ל-24.7, יוצא מעל 24.9

בחניה 0 ההבדל בין רכב נמוך לחניה ריקה הוא רק כ-0.5 ס"מ, ולכן המרווח מכל צד של הספים צר מאוד (כ-0.2 ו-0.1 ס"מ). אם הערכים שלכם שונים — עדכנו את הקבועים (*Programmer Manual*, סעיף 2.2).

---

9. פתרון תקלות

תופעה	סיבה אפשרית	פתרון
אין פורט ברשימת **Port**	דרייבר CH9102 חסר	התקינו את הדרייבר
ההעלאה נתקעת על "Connecting..."	הלוח אינו במצב העלאה	החזיקו **BOOT** בתחילת ההעלאה
`parking_page.h: No such file`	הקובץ אינו באותה תיקייה	העבירו אותו לתיקיית ה-`.ino`
ג'יבריש ב-Serial Monitor	קצב שגוי	**baud 9600**
הפקודות לא נשלחות	אין תו סוף שורה	**New Line**
`[CHECK]` מציג `--`	אין הד	ודאו שהחיישן פונה למשטח מוצק
רכבים נוצרים מעצמם	הקיר מול חיישן הכניסה קרוב מ-25 ס"מ, או משהו עובר מולו	ודאו שהכביש הריק נותן ~30 ס"מ ב-`check`
חניה 0 "קופצת" בין תפוסה לפנויה	הקריאות קרובות לספים (24.7/24.9)	`check 10`, והזיזו את `SPOT_ARRIVE_CM` / `SPOT_LEAVE_CM` או הגביהו את החיישן
חניה 0 מסומנת `BLOCKED`	החיישן רואה עצם ואין רכב מוקצה	הסירו את העצם
רשת `SmartParking` לא מופיעה	ה-WiFi לא עלה	בדקו את שורת `[WEB]`; הפעילו את הלוח מחדש
העמוד תקוע על "connecting"	כתובת שגויה / לא מחוברים לרשת	התחברו ל-`SmartParking` והשתמשו בכתובת מ-`[WEB]`

10. מגבלות שכדאי להכיר

- **כיבוי הלוח מאפס את החניון.**
- **רק חניה 0 מזוהה אוטומטית**; לשאר החניות משתמשים ב-`park` ו-`leave`.
- **אין זיהוי רכב אמיתי.** רכב שמגיע לחניה 0 משויך לפי השריון, ואם אין — לרכב האחרון שטרם חנה.
- **העמוד הגרפי לצפייה בלבד.** כל השליטה דרך ה-Serial Monitor.
- **לאחר 500 רכבים** נדרשת הפעלה מחדש של הלוח.